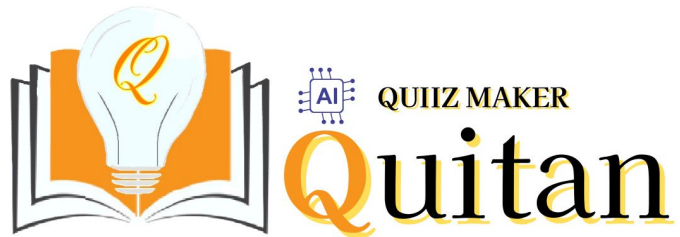


QUIIZ MAKER

Quitan

生成AIで作って遊べる
クイズアプリ



生成AIでクイズを生成するアプリ

カテゴリやキーワードを指定するだけで簡単にAIがクイズ生成してくれます。

生成したクイズはデータベースに保存され、自分以外のユーザーが生成したクイズにもチャレンジできます。また、カテゴリやキーワードでクイズを検索して興味のある分野でのクイズをお楽しみいただけます。

登録ユーザー用の機能として、お気に入りの問題や間違えた問題のみを絞り込む機能が検索機能に加わります。

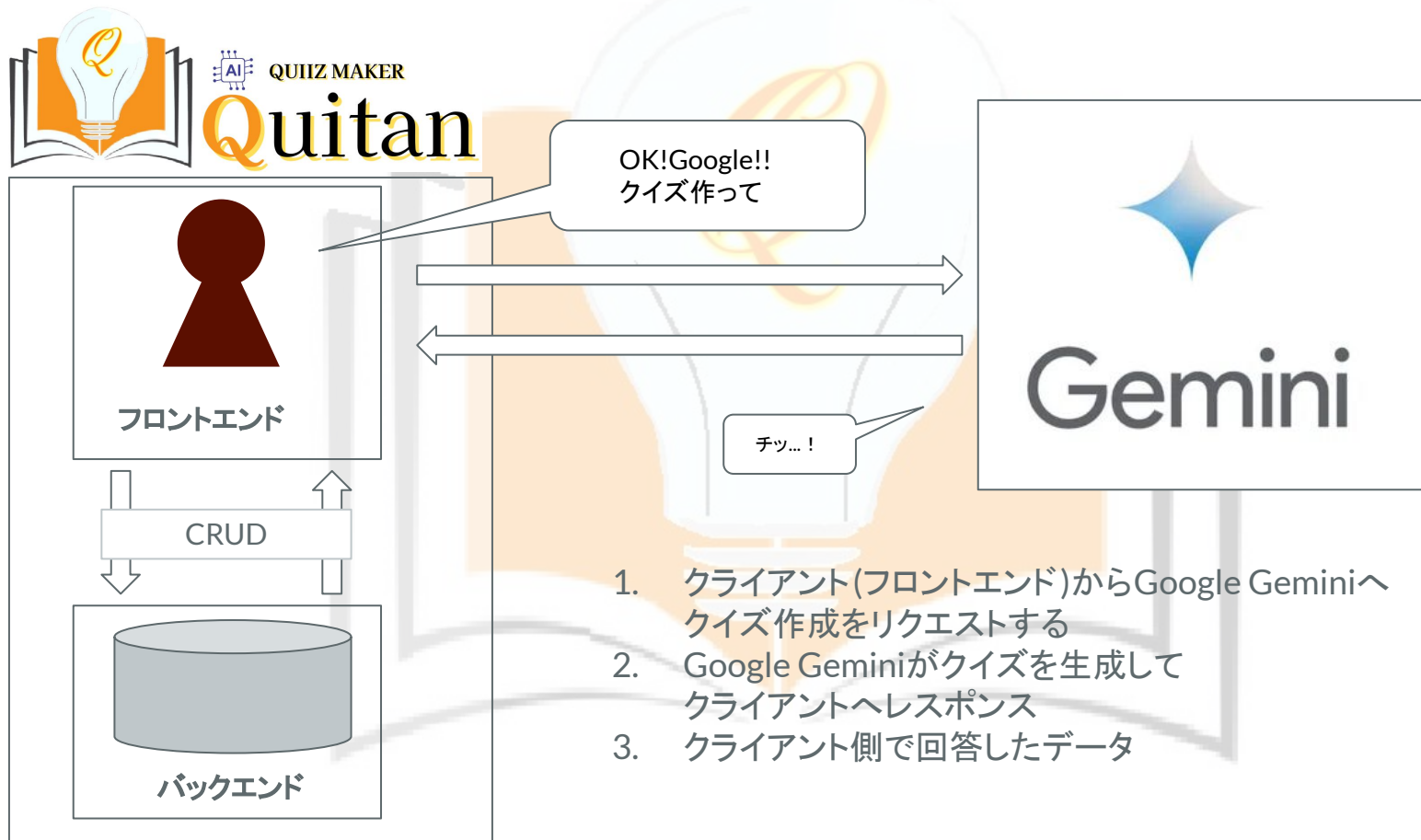
発表の流れ

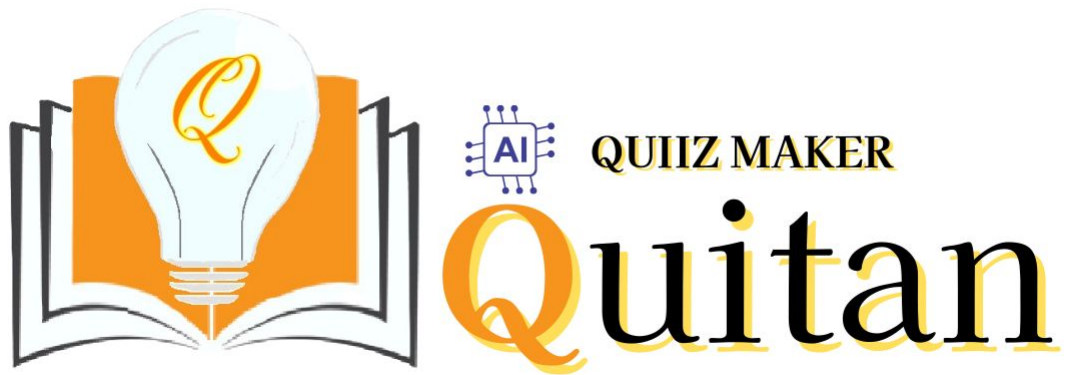
- 使用技術について
 - 使用技術一覧
 - CSSフレームワーク
 - CDNとWeb-API
 - 2つのWeb-API(Gemini-API)
 - 2つのWeb-API(REST-API)
- 設計紹介
 - データベース設計
 - データ遷移図
 - エンドポイント
 - クラス図
- 実演

—— メンバー ——

L: 山福 他(50音順): 須川、銅子、中島

アプリケーション概要





使用技術 について

- 使用技術一覧
- CSSフレームワーク
- CDNとWeb-API
- 2つのWeb-API(Gemini-API)
- 2つのWeb-API(REST-API)

使用技術について

<CSSフレームワーク>

Bootstrap

<CDN>

JQuery: Bootstrapや各CDNの補助

axios: RESTによるデータの送受信

awesomplete: キーワードのサジェスト機能

JavaScriptライブラリ

<Web-API>

Gemini-API: テキスト生成のapiをクイズ生成に利用

REST-API: JavaScriptとSpring間のデータ通信に利用(Springにエンドポイント作成)

CSSフレームワーク「Bootstrap」

<CSSフレームワークとは>

ウェブサイトやウェブアプリケーションのスタイリングを効率的に行うための事前定義されたCSSの集合体。

【複数人でフロントエンド開発をするときの難点】

- 各自がそれぞれ思惑でクラスやスタイルの設定をすると、レイアウトがバラける
- スタイル設定同士の競合が起きて、思った通りにスタイルが適用されない

あらかじめ用意された
CSSファイル、JSファイルを皆で使ってスタイルのバラつきを抑える

【メリット】

- スタイルの統一
- 開発の効率化
- スマホ対応が楽(レスポンシブデザイン)
- JSの簡易実装(モーダルやアコーディオン等)
- メンテナンス性の向上

CSSフレームワーク「 Bootstrap 」

<使用方法>

公式サイトから「CSSファイル」「JSファイル」をダウンロードする

<https://getbootstrap.jp/docs/5.3/getting-started/download/>

【さらに楽する方法】

テンプレートサイトからBootstrap で作成されたサイトのDLが可能

テンプレートサイト: <https://themewagon.com/>

参考テンプレート: <https://themewagon.github.io/kaiadmin-lite/index.html>

<初期レイアウト: Figma>

<https://www.figma.com/design/rGF5siteI1bgEKKXT1bXej/%E3%82%AF%E3%82%A4%E3%82%BA%E3%82%A2%E3%83%97%E3%83%A2%E3%83%97%E3%83%A4?node-id=11-7642&t=gJ2T7V2352811Lue-1>

CDNとWeb-API

CDN: Content Delivery Network

API: Application Programming Interface

【共通点】

他の人が作成したプログラムWeb経由で取得して
自身のアプリケーションに組み込む

(CDN)

静的コンテンツの利用

内容が確定している
コンテンツを取得する

- CSSファイル
- JSファイル

(API)

動的コンテンツの利用

状況で内容が変化する
コンテンツを取得する

- 商品紹介ブログ等に
貼られているAmazonのリンク

2つのWeb-API(Gemini-API)

【Gemini-API】

JavaScriptからpromptのデータを送信するとpromptの内容に沿ったテキストがresponseデータとして返ってくる

＜今回のクイズアプリでの利用＞

クイズを生成するためのテンプレートをpromptに記述してGemini-APIに送信

(テンプレートのへ記載内容)

- タイトル、質問
選択肢、解説
を作成する指示とその文字数制限
- カテゴリー、キーワードの変数

(参考) Gemini-api クイックスタート

<https://ai.google.dev/gemini-api/docs/quickstart?hl=ja&lang=rest>

```
const prompt = `
カテゴリ: ${selectedCategoryName}
キーワード: ${sendKeywords}

(条件)
下記のテンプレートに従ってクイズを作成してください。
問題は200文字～400文字で作成してください。
回答は4択で1番目を正解の選択肢にしてください。
回答は正解の選択肢のみに前後に【**】がついてしまうなどの
一目で解答がわかってしまうような作成の仕方はしないでください。
また、【問題】の後の問題文と【選択肢】の後の選択肢一覧に改行による空白が入らないようにしてください。
カテゴリとキーワードに関連性がない場合等で問題が生成できない場合は【失敗】と返してください。
<以下テンプレート>
【カテゴリ】
選択したカテゴリが入ります。
【タイトル】
問題のトピックが入ります。(できるだけ具体的に40文字以内で※文字数制限厳守)
【問題】
問題文が入ります。(200文字～400文字※絶対に400字を超えない)
少しマニアックな方が良く、また、質問内容はわかりやすく簡潔に
【選択肢】(各100文字以内厳守)
1: 正解の選択肢
2: 誤りの選択肢
3: 誤りの選択肢
4: 誤りの選択肢
【解説】
解説が入ります。(詳しい補足情報を入れる。解説に選択肢番号は入れない。)
【以上】
<以上テンプレート>

try {
```

2つのWeb-API(REST-API)

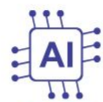
【REST-API】

アプリケーションで設定したエンドポイントにアクセスしてデータを送受信する
JavaScriptとSpring間のデータ連携に必要な



<今回のクイズアプリでの利用>

- GET: キーワード一覧の取得 → キーワード入力のサジェストに利用
- GET: カテゴリー一覧の取得 → クイズ生成時のカテゴリー指定を DB内のものに限定
- POST: 生成したクイズのデータをバックエンド (Spring) に送信



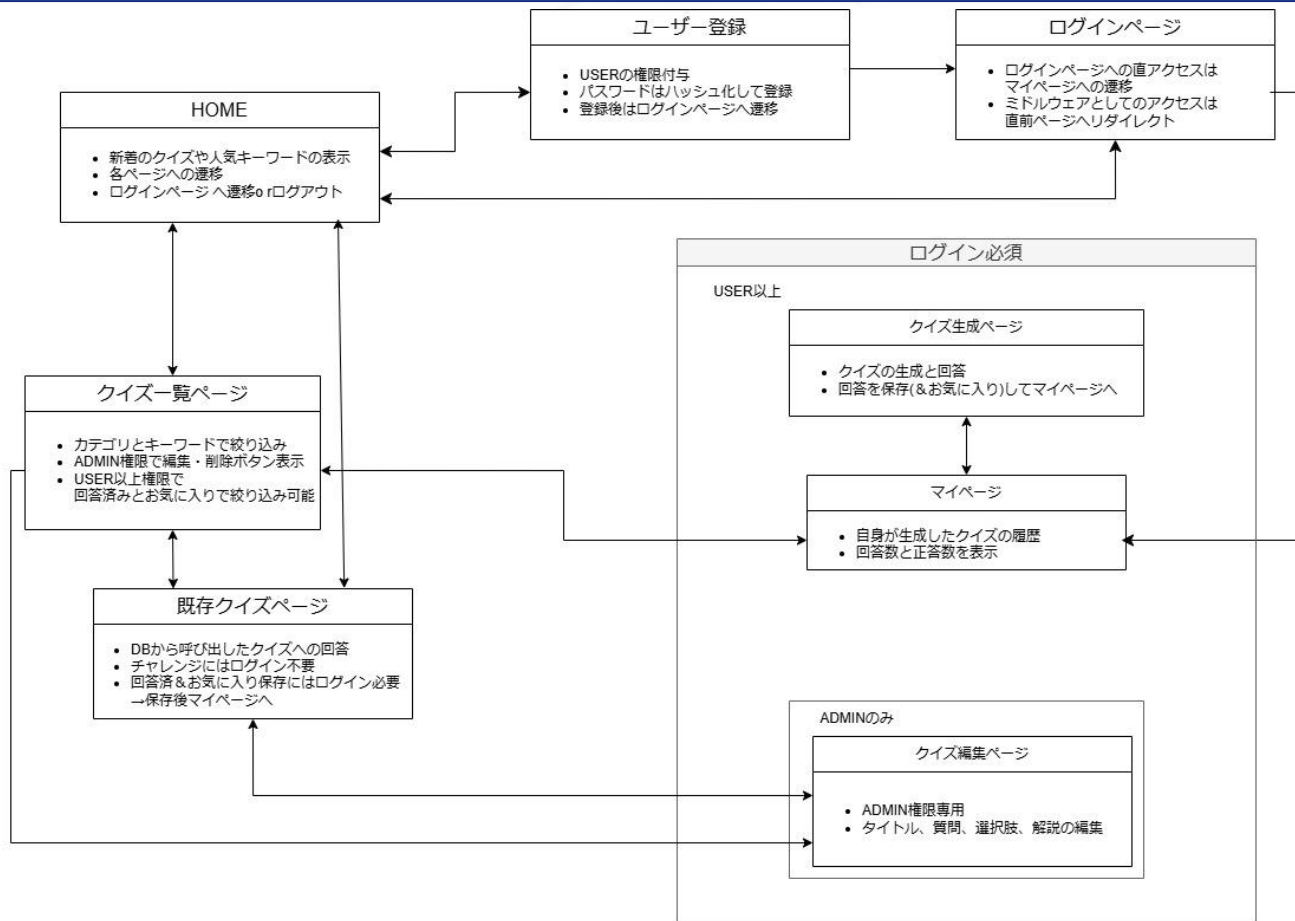
QUIIZ MAKER

Quitan

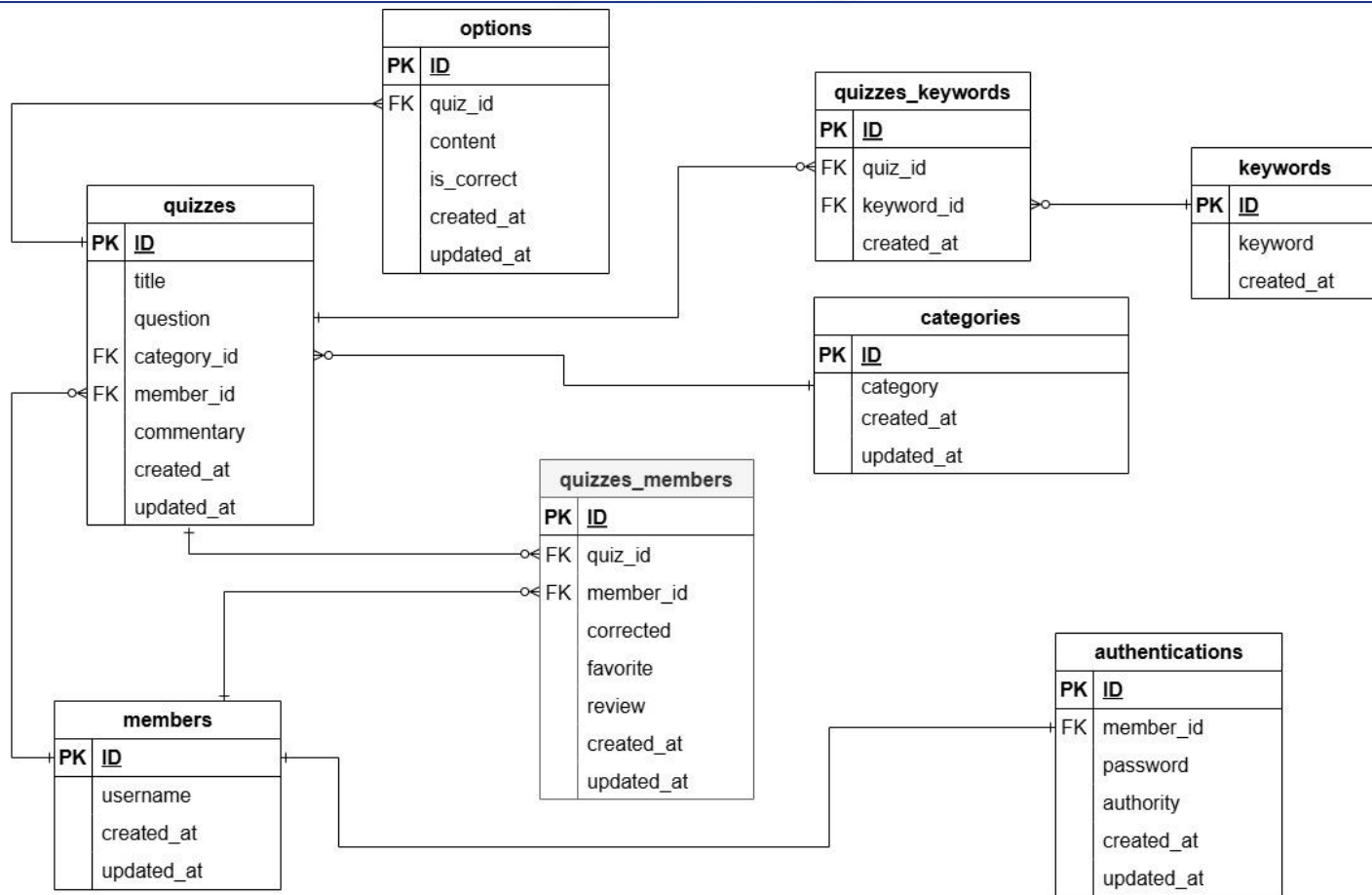
設計紹介

- ページ遷移
- データベース設計
- エンドポイント設計
- データ遷移
- クラス図

ページ遷移図



データベース設計 (ER図)



データベース設計 (テーブル)

テーブル名	カラム名	型	キー	外部キー	NULL
quizzes	id	SERIAL	PK		NOT
	title	VARCHAR(40)			NOT
	question	VARCHAR(1000)			NOT
	category_id	INTEGER	FK	categories(id)	NOT
	member_id	INTEGER	FK	members(id)	NOT
	commentary	VARCHAR(1000)			NOT
	created_at	TIME_STAMP			NOT
	updated_at	TIME_STAMP			
options	id	SERIAL	PK		NOT
	quiz_id	INTEGER	FK	quizzes(id)	NOT
	content	VARCHAR(200)			NOT
	is_correct	BOOLEAN			NOT
	created_at	TIME_STAMP			NOT
	updated_at	TIME_STAMP			
keywords	id	SERIAL	PK		NOT
	keyword	VARCHAR(40)			NOT
	created_at	TIME_STAMP			NOT
quizzes_keywords	id	SERIAL	PK		NOT
	quiz_id	INTEGER	FK	quizzes(id)	NOT
	keyword_id	INTEGER	FK	keywords(id)	NOT
	created_at	TIME_STAMP			NOT
categories	id	SERIAL	PK		NOT
	category	VARCHAR(40)			NOT
	created_at	TIME_STAMP			NOT
	updated_at	TIME_STAMP			

テーブル名	カラム名	型	キー	外部キー	NULL
members	id	SERIAL	PK		NOT
	username	VARCHAR(40)			NOT
	created_at	TIME_STAMP			NOT
	updated_at	TIME_STAMP			
quizzes_members	id	SERIAL	PK		NOT
	quiz_id	INTEGER	FK	quizzes(id)	NOT
	member_id	INTEGER	FK	members(id)	NOT
	corrected	BOOLEAN			NOT
	favorite	BOOLEAN			NOT
	review	VARCHAR(400)			
	created_at	TIME_STAMP			NOT
	updated_at	TIME_STAMP			
authentications	id	SERIAL	PK		NOT
	member_id	INTEGER	FK		NOT
	password	VARCHAR(50)			NOT
	authority	role			NOT
	created_at	TIME_STAMP			NOT
	updated_at	TIME_STAMP			

エンドポイント設計

	メソッド	エンドポイント	権限	分類	処理	テンプレ	リダイレクト	備考
quizController	GET	/		画面遷移	トップページを表示	index		
	GET	/quiz/list?category?keywords?favorite?corrected		画面遷移	クイズリストページを表示	list		権限で要素切替あり
	GET	/quiz/{id}		画面遷移	クイズページを表示	quiz		
	POST	/answered		SQL更新	クイズの回答状況を登録	-	my-page	
	GET	/quiz/edit/{id}	ADMIN	画面遷移	クイズ編集ページを表示	edit		
	POST	/quiz/update	ADMIN	SQL更新	クイズ情報を更新	-	list	
	POST	/quiz/delete/{id}	ADMIN	SQL更新	クイズ情報を削除	-	list	
	GET	/quiz/generate	USER	画面遷移	クイズ生成ページを表示	generate		
quizApiController	POST	/api/quiz-create	USER	api(SQL更新)	クイズを新規登録	-	my-page	
	GET	/api/keywords		api(SQL取得)	キーワード一覧をJSON取得	-		
	GET	/api/categories		api(SQL取得)	カテゴリー一覧をJSON取得	-		
	GET	/api/options/{quizId}		api(SQL取得)	選択肢をJSON取得	-		
memberController	GET	/my-page{id}	USER	画面遷移	マイページを表示	mypage		
authenticateController	GET	/signup		画面遷移	ユーザー登録ページを表示	register		
	POST	/signup		SQL更新	ユーザー登録処理	-	login	
	GET	/login		画面遷移	ログインページの表示	login		

DATA FLOW

クライアント

アプリケーション

データベース

新着クイズ呼び出し、他

utilityComponent

quizNum
categoryNum
keywordNum

List<Quiz>

Quiz

id

title

Category

List<Keyword>

List<Keyword>

Keyword

id

keyword

List<Category>

Category

id

category

selectNewQuiz

連結

quizzes

quizzes_keywords

keywords

categories

selectPopularKeywords

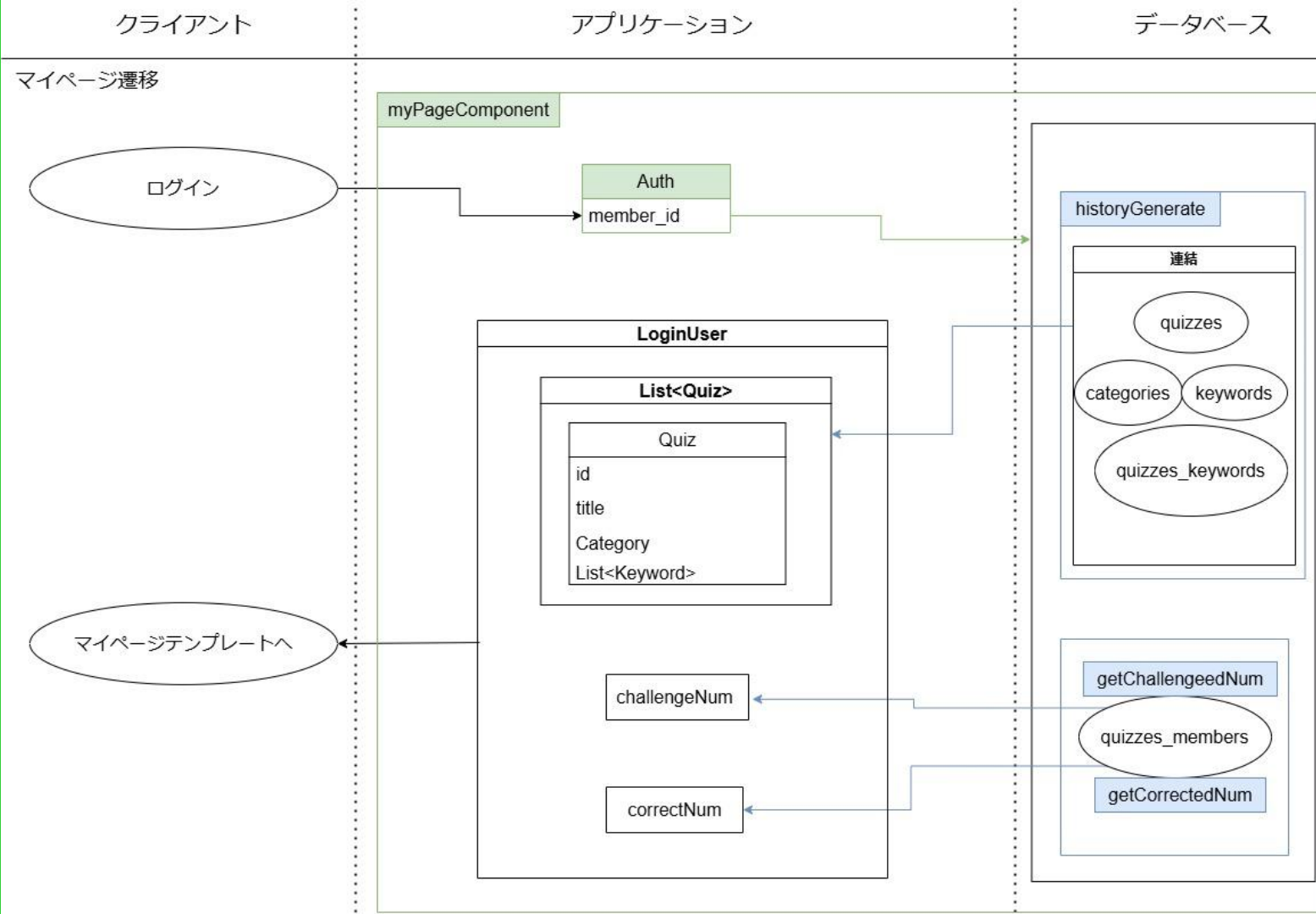
keywords

selectPopularCategories

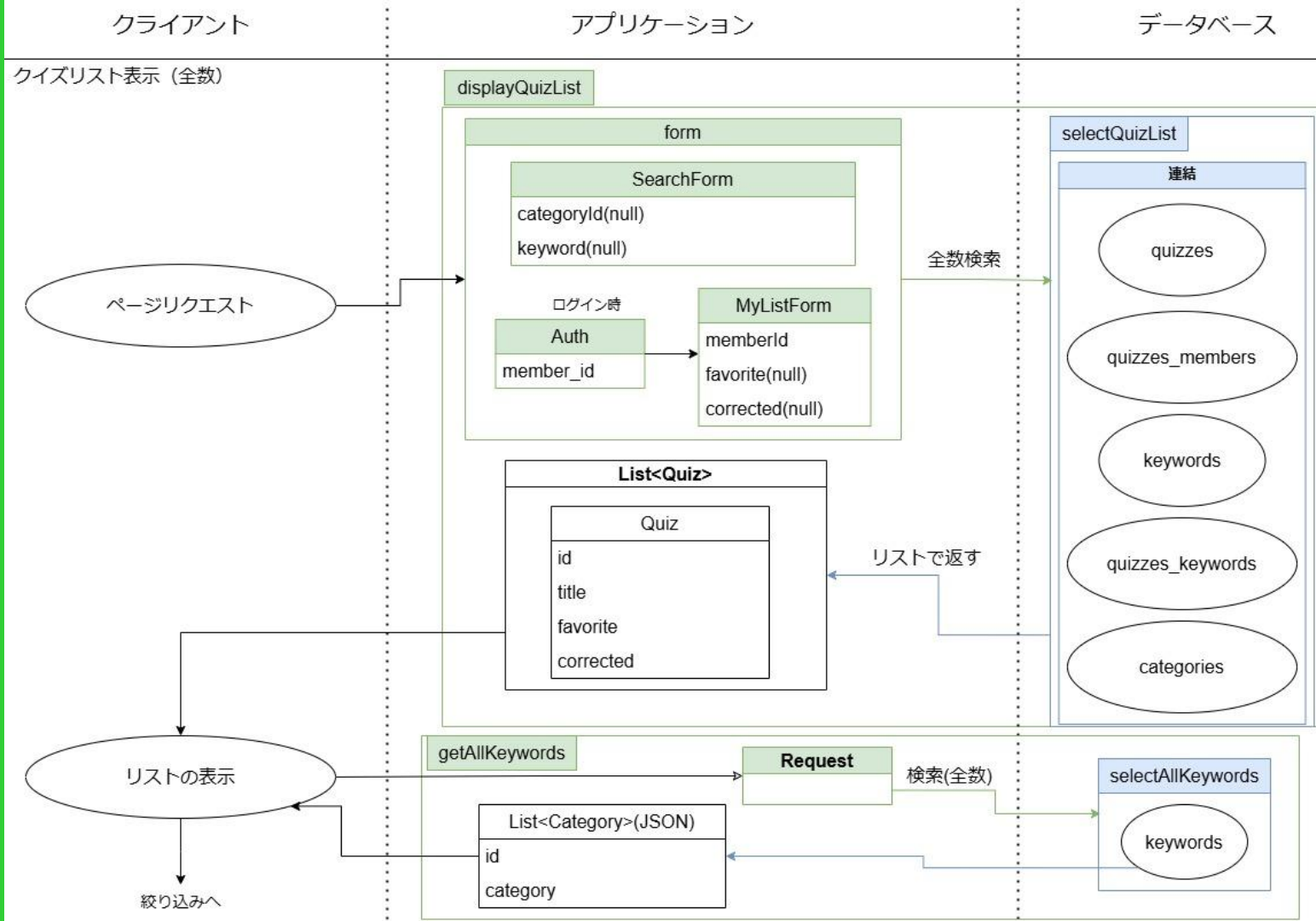
categories

各ページテンプレートへ

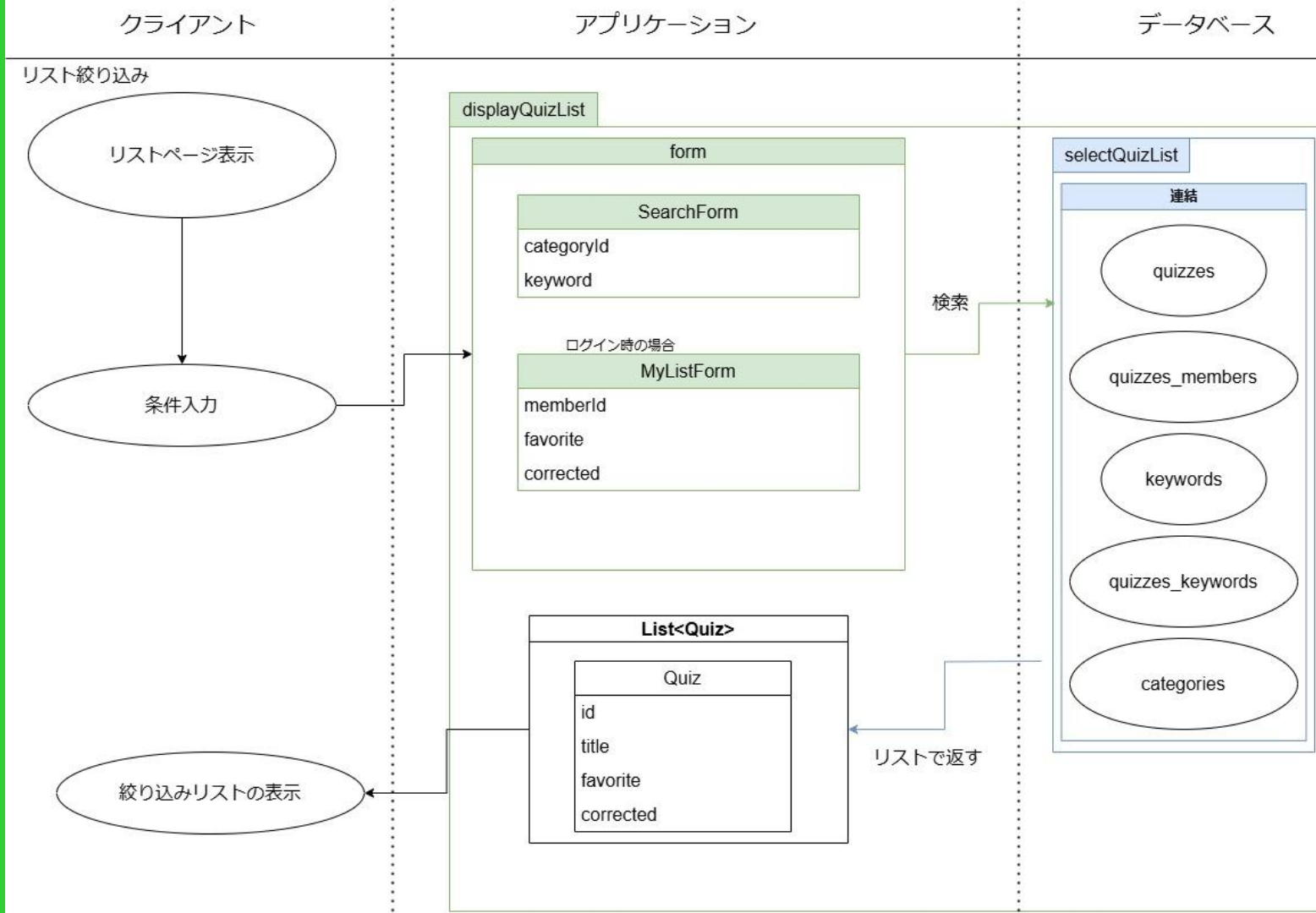
DATA FLOW



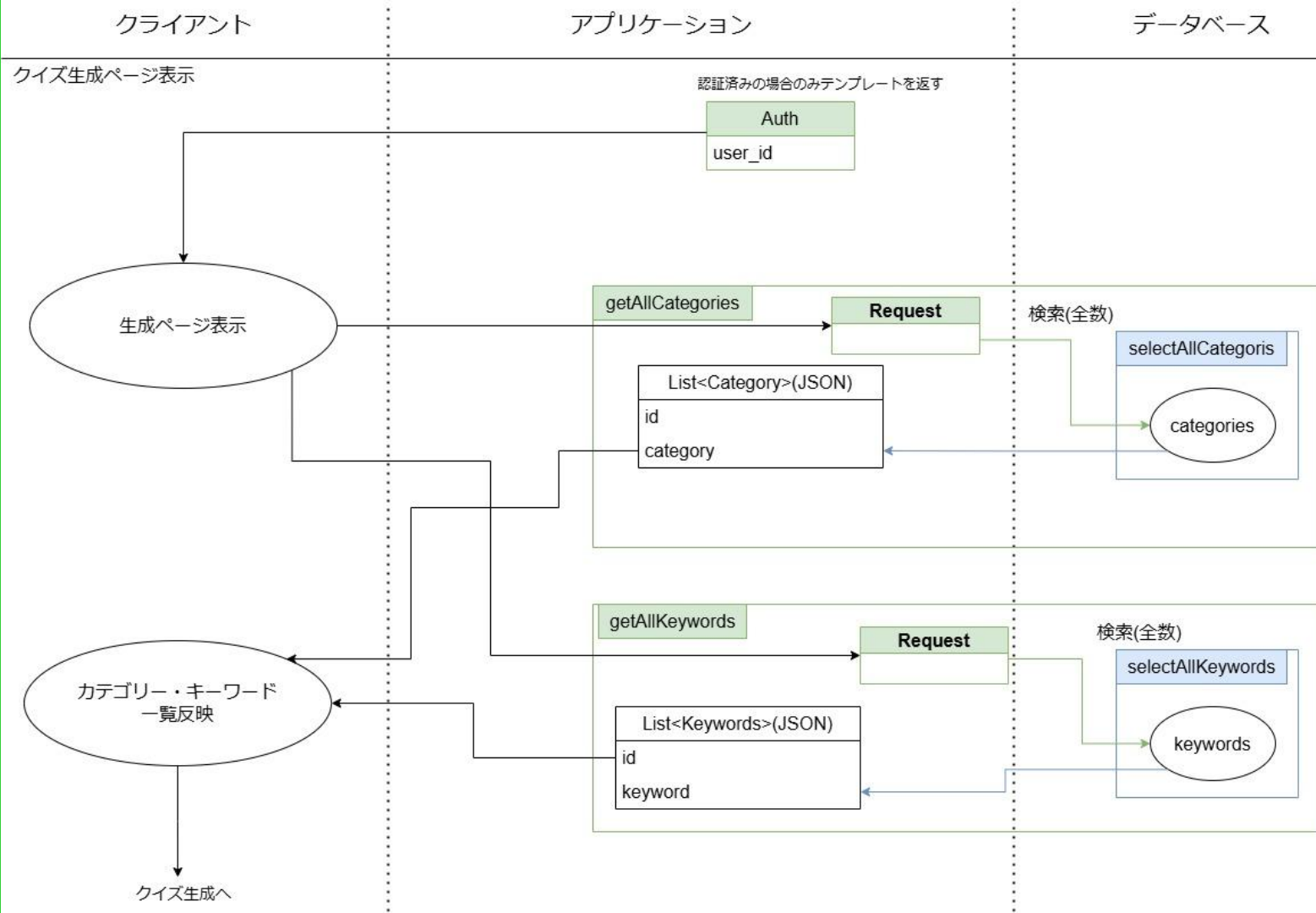
DATA FLOW



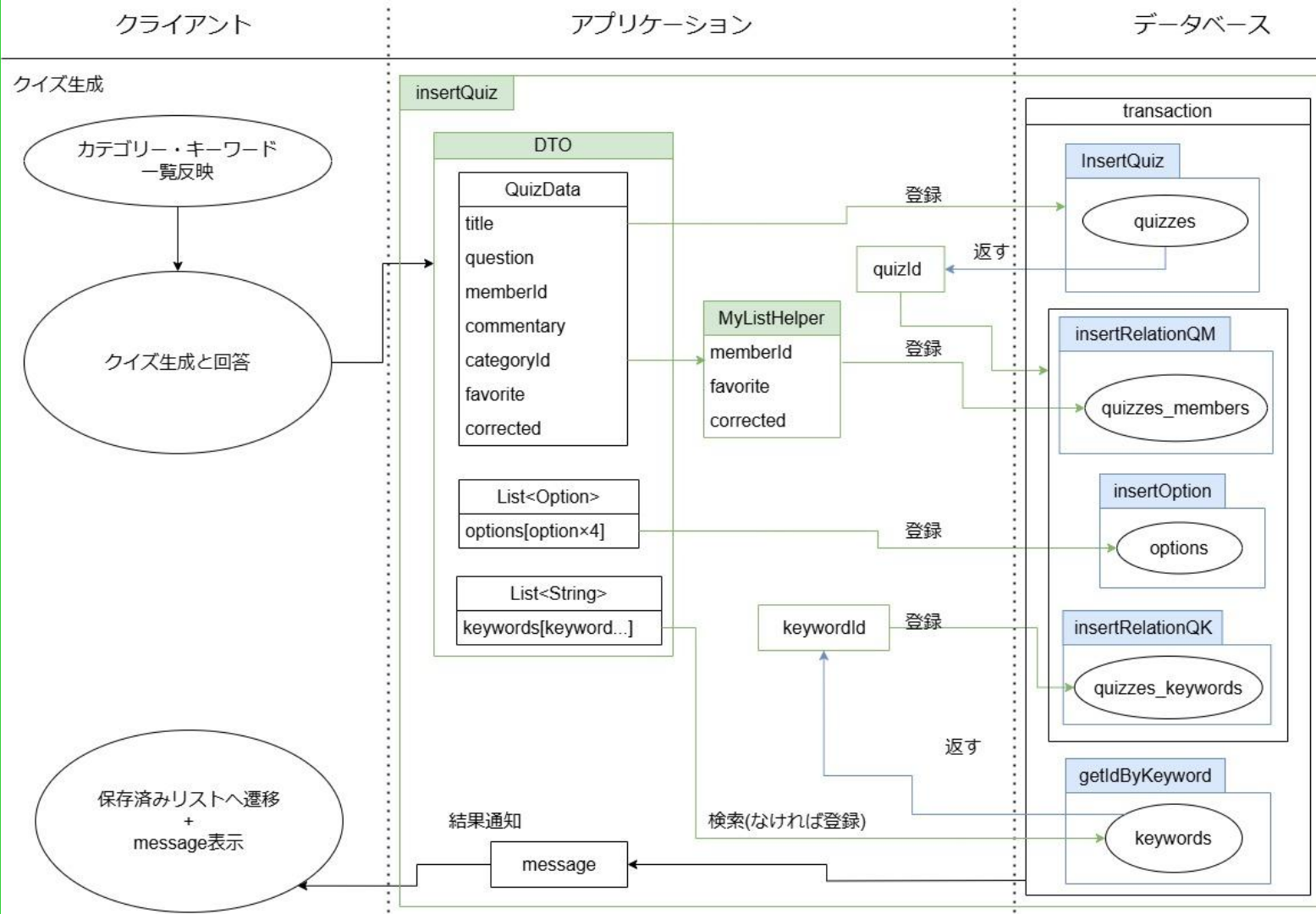
DATA FLOW



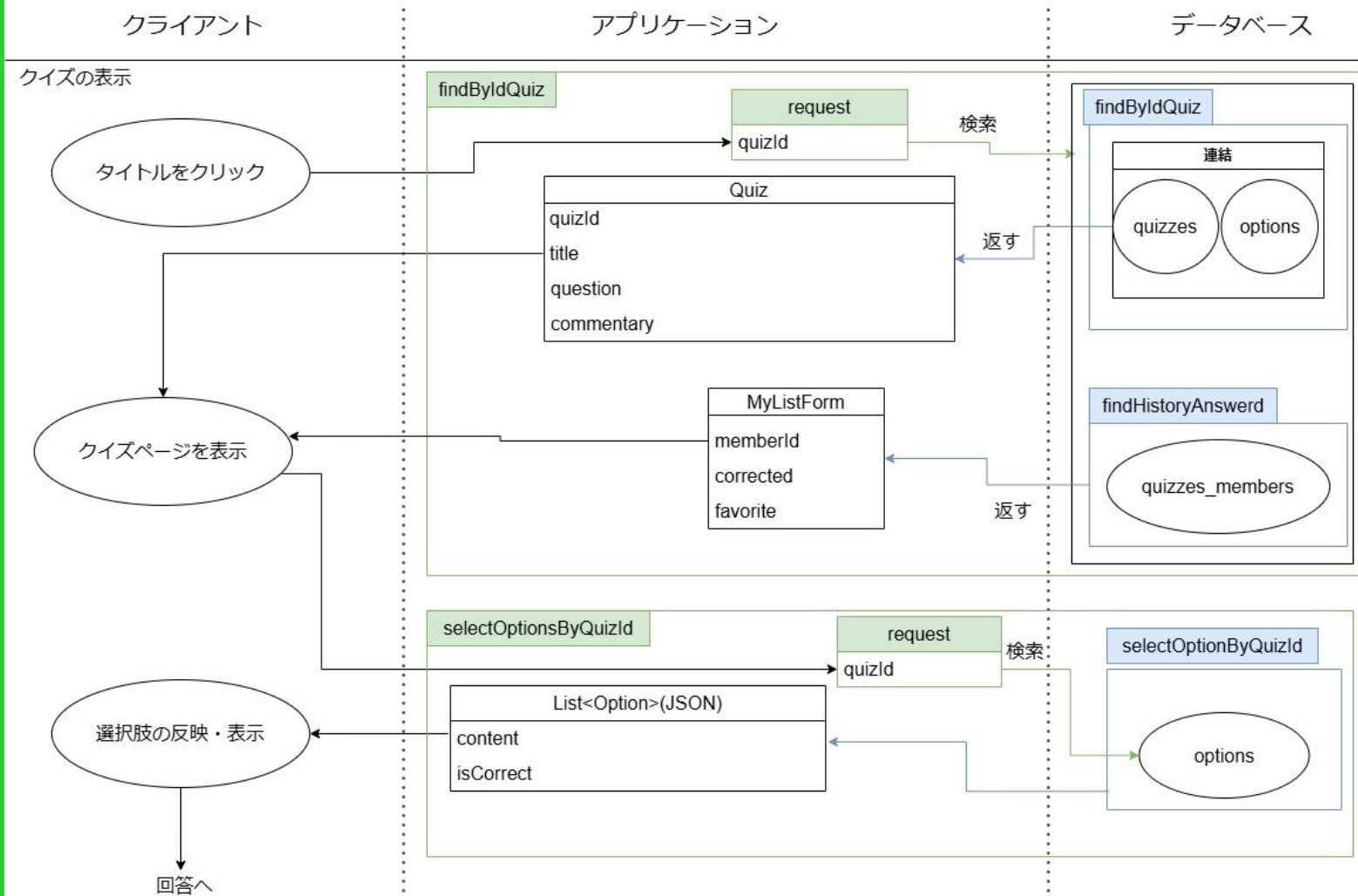
DATA FLOW



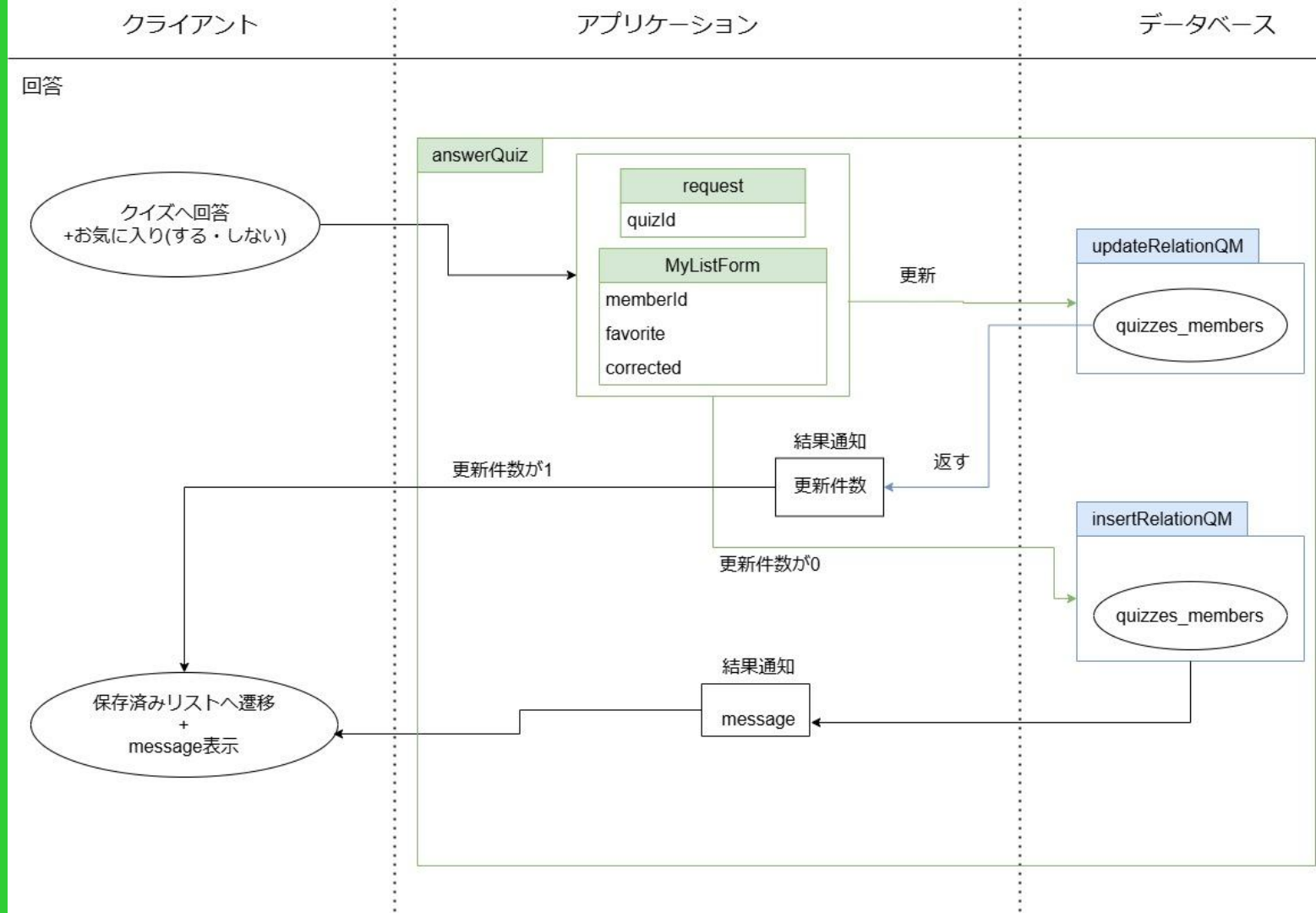
DATA FLOW



DATA FLOW



DATA FLOW



DATA FLOW

クライアント

アプリケーション

データベース

クイズの表示(更新用・ADMINのみ)

findByIdQuiz

検索

findByIdQuiz

リストから
編集ボタンクリック

request

quizId

返す

連結

quizzes

options

form

QuizForm

id
title
question
commentary

Quiz

quizId
title
question
commentary

List<Options>

id
content
isCorrect

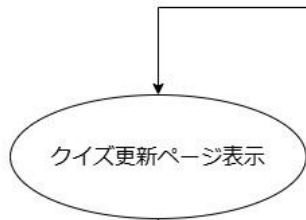
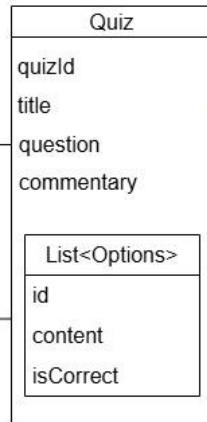
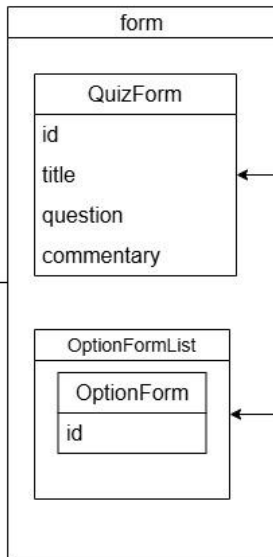
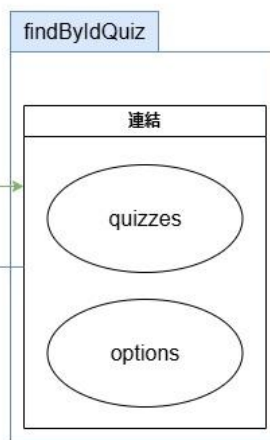
OptionFormList

OptionForm

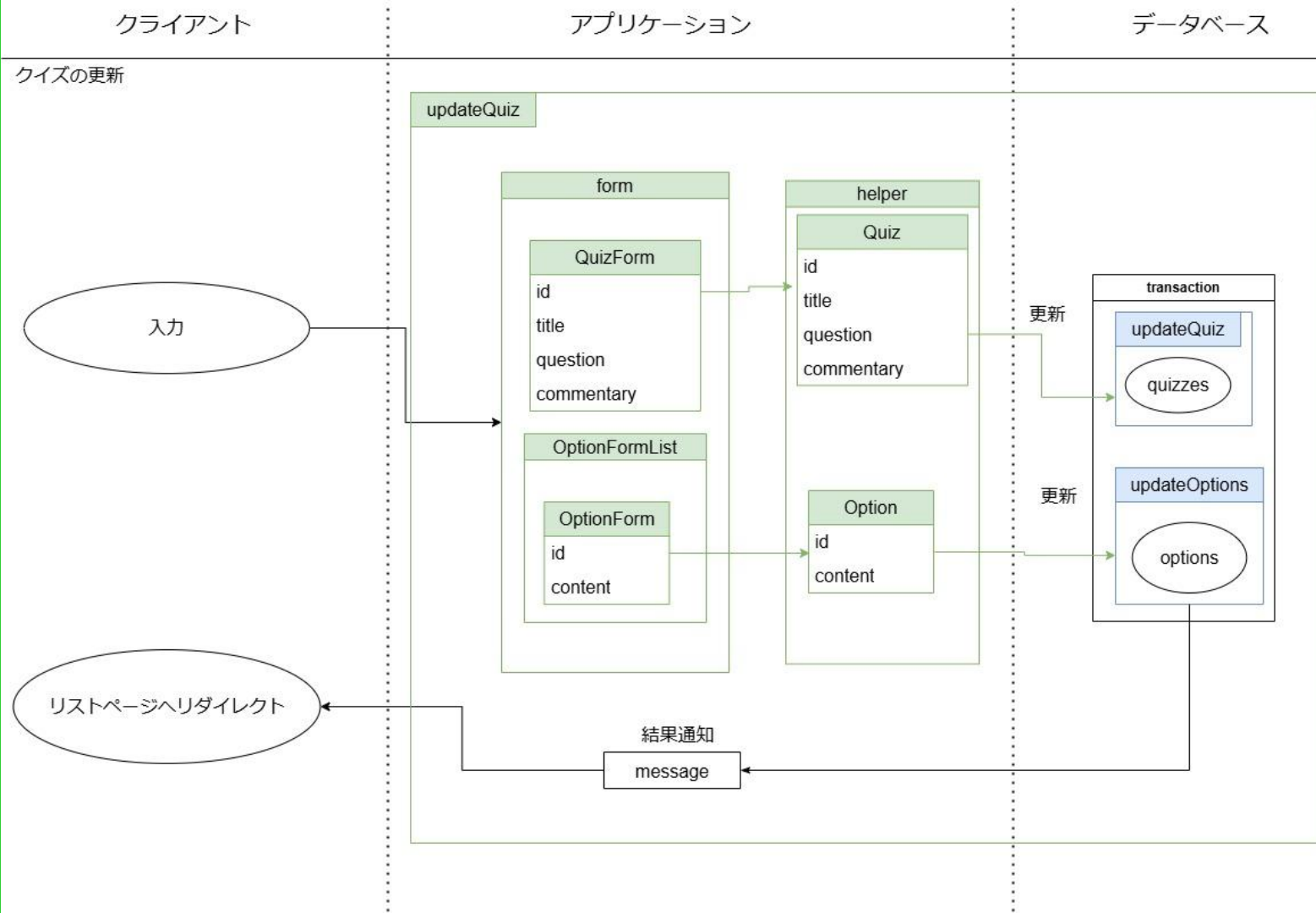
id

クイズ更新ページ表示

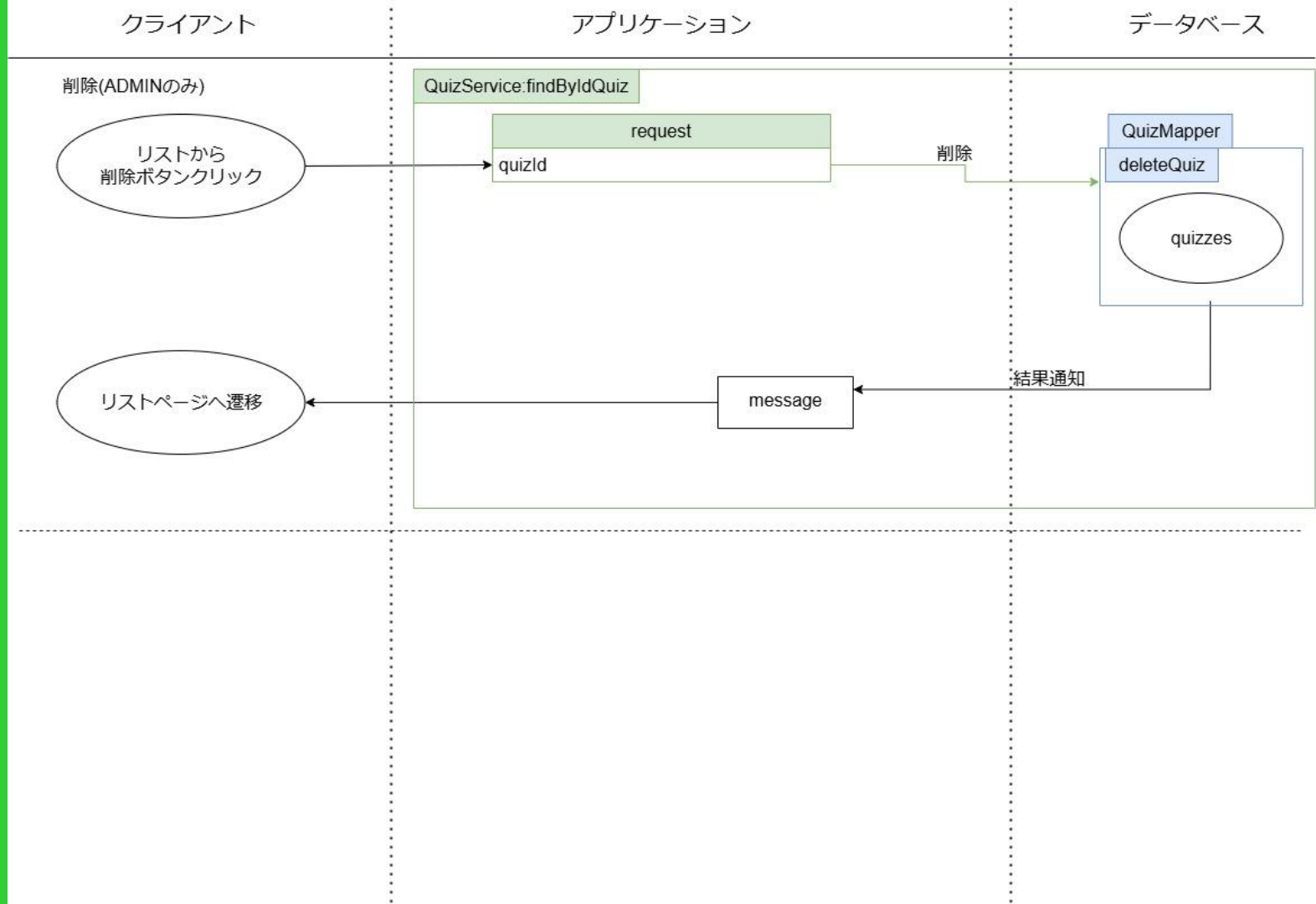
更新へ



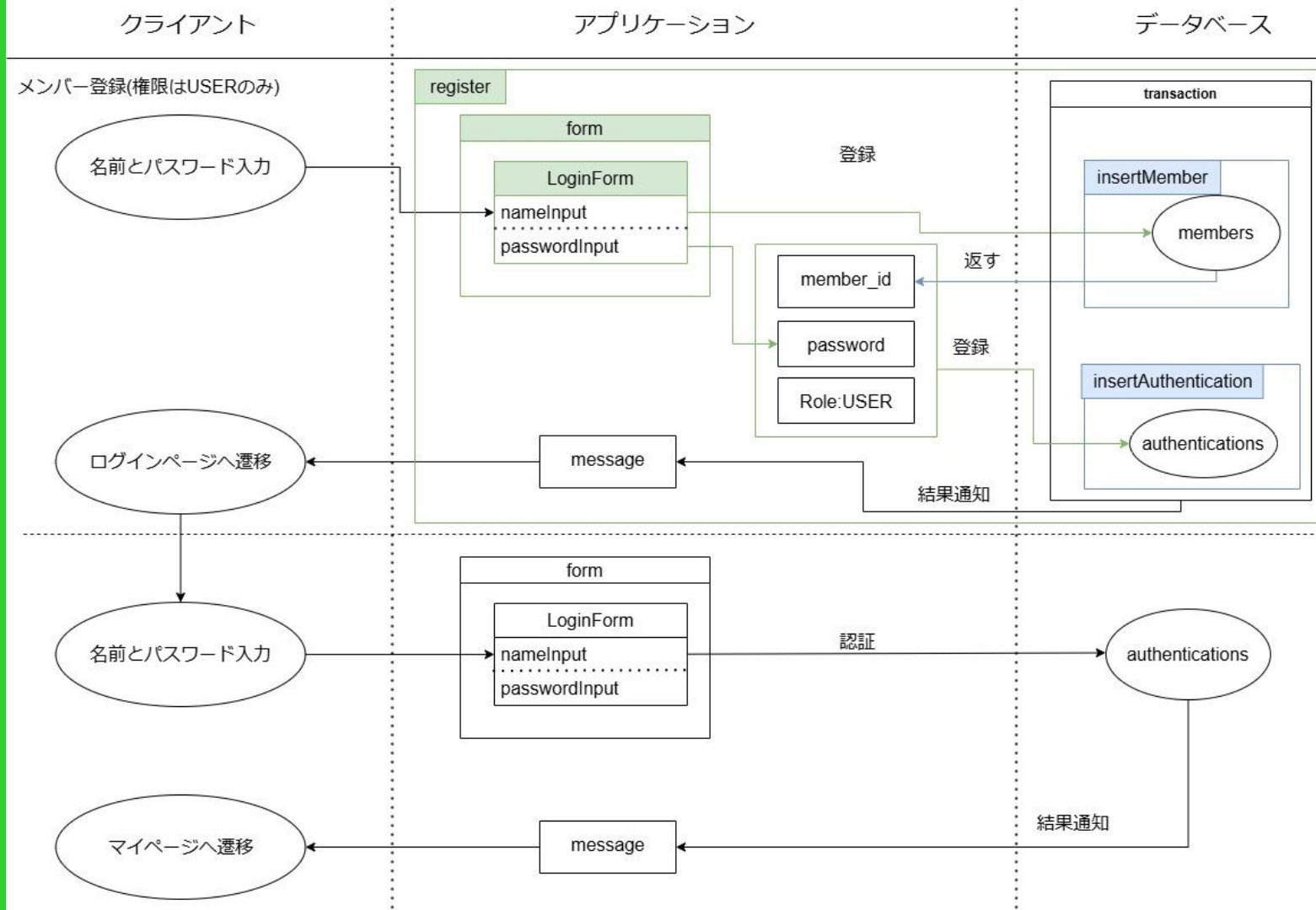
DATA FLOW



DATA FLOW



DATA FLOW



CLASSES

entity

Quiz
int id
String title
String question
Category category
Member createMember
List<Keyword> keywords
List<Option> options
String commentary
Boolean favorite
Boolean corected
LocalDateTime createdAt
LocalDateTime updatedAt

Options
int id
int quizId
String content
Boolean isCorrect
LocalDateTime createdAt
LocalDateTime updatedAt

Category
int id
String category
LocalDateTime createdAt
LocalDateTime updatedAt

Keyword
int id
String keyword
LocalDateTime created_at

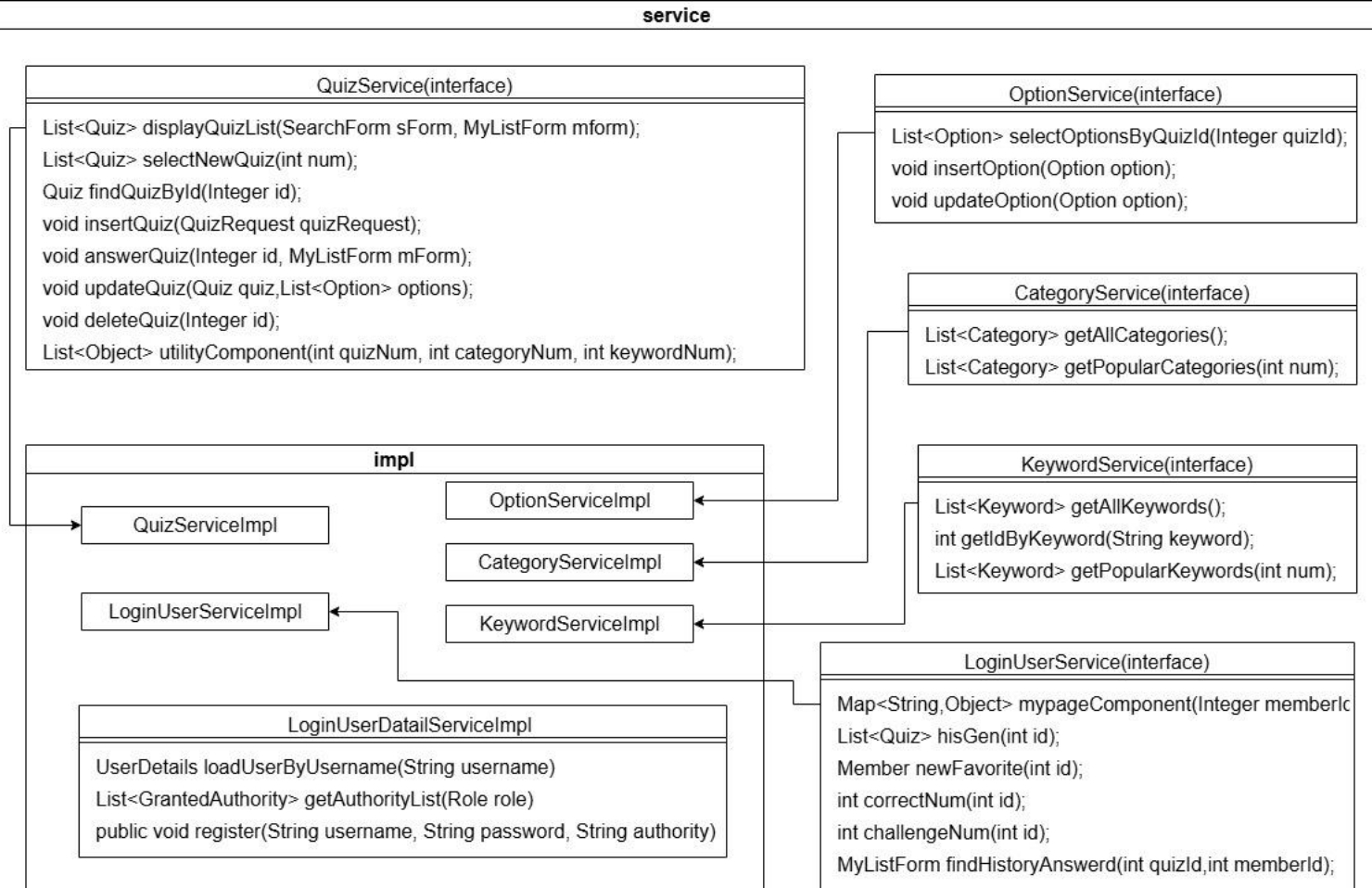
Member
int id
String username
List<Quiz> quizzes
LocalDateTime createdAt
LocalDateTime updatedAt

Authentication
int id
Member member
String password
Role authority
LocalDateTime createdAt
LocalDateTime updatedAt

LoginUser(ex: User)
int userId
LoginUser(引数3つ){}

Role(ENUM)
ADMIN
USER

CLASSES



CLASSES

repository

QuizMapper

```
List<Quiz> selectQuizList(@Param("sForm") SearchForm sForm, @Param("mForm") MyListForm mForm);
List<Quiz> selectNewQuiz(int num);
Quiz findQuizById(@Param("id") Integer id);
void insertQuiz(@Param("quizData") QuizRequest.QuizData quizData);
void answerQuiz(Integer id, MyListForm mForm);
void updateQuiz(Quiz quiz);
void deleteQuiz(@Param("id") Integer id);
void insertRelationQK(@Param("quizId")int quizId, @Param("keywordId") int keywordId);
void insertRelationQM(@Param("quizId") Integer quizId, @Param("mform") MyListForm mform);
int updateRelationQM(@Param("quizId") Integer quizId, @Param("mform") MyListForm mform);
List<Quiz> historyGenerate(int memberId);
```

KeywordMapper

```
List<Keyword> selectAllKeywords();
List<Keyword> selectKeywordsByQuizId(Integer quizId);
Integer getIDByKeyword(@Param("keyword") String keyword);
void insertKeyword(@Param("keyword") Keyword keyword);
List<Keyword> selectPopularKeywords(int num);
```

OptionMapper

```
List<Option> selectOptionsByQuizId(Integer quizId);
void insertOption(@Param("option") Option option);
void updateOption(@Param("option") Option option);
void deleteOption(@Param("quizId") Integer quizId);
```

CategoryMapper

```
List<Category> selectPopularCategories(int num);
List<Category> selectAllCategories();
```

AuthenticationMapper

```
Authentication selectByUsername(String username)
void insertAuthentication(Authentication authentication)
```

MemberMapper

```
void insertMember(Member member)
void updateMember(Member member)
void deleteMember(@Param("id") Integer id);
int getCorrectedNum(@Param("id") Integer id);
int getChallengeedNum(@Param("id") Integer id);
Member getNewFavoriteQuiz(@Param("id") Integer id);
MyListForm findHistoryAnswerd(@Param("quizId") Integer quizId, @Param("memberId") Integer memberId );
```

CLASSES

form

QuizForm
int id
String title
String question
String commentary

MyListForm
Integer memberId
Boolean favorite
Boolean corrected

LoginForm
String nameInput
String passwordInput

OptionForm
int id
String content

OptionFormList
List<OptionForm> oForms;

SearchForm
Integer categoryId
String keyword

helper

QuizHelper
static Quiz convertQuiz(QuizForm form){}
static QuizForm convertQuizForm(Quiz quiz){}

OptionHelper
static List<Option> convertToOption(List<OptionForm> oForms)
static List<OptionForm> convertToOptionForms(List<Option> options)

MyListHelper
static MyListForm convertToMyList(int memberId, boolean corrected, boolean favorite)

DTO

QuizRequest
QuizData quizData
List<Option> options
List<String> keywords
static class QuizData(内部クラス)

config

SecurityConfig
final UserDetailsService userDetailsService
final PasswordEncoder passwordEncoder
final CustomAuthenticationSuccessHandle
SecurityFilterChain filterChain(HttpSecurity http)

PasswordConfig
PasswordEncoder passWordEncoder()

PasswordConfig
CustomAuthenticationSuccessHandler

实践

